

two classes. This makes it much easier to program each piece, since the scope of work is smaller and the responsibilities are clearer for each. Further, having only a single global variable ensures that lower-level code will not accidentally affect data at the global level.

In computer literature, the construct shown in [Figure 4-1](#) is often known as *composition* or *object composition*.

composition

A logical structure in which one object manages one or more other objects

You can think of one object as being made up of other objects. An example is that a car object is made up of an engine object, a steering wheel object, some number of door objects, four wheel and tire objects, and so on. The discussion often centers around the relationships between objects. In this example, one would say that a car “has a” steering wheel, an engine, some number of doors, and so on. Therefore, the car object is a composite of other objects.

We’ll have three separate files. The main code lives in its own file. It imports the code of our new *Bank.py* file that contains the `Bank` class ([Listing 4-7](#)). The `Bank` class imports the code of the *Account.py* file and uses it to instantiate `Account` objects as needed.

Building the Object Manager Object

[Listing 4-7](#) shows the code of the new `Bank` class, which is an object manager object.

File: BankOOP5_SeparateBankClass/Bank.py

```
# Bank that manages a dictionary of Account objects

from Account import *

class Bank():
```